



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DESIGN OF AN AI-BASED INTELLIGENT CAMPUS HELPDESK AND FACILITY FAULT-DIAGNOSIS ASSISTANT

CSA1710 – Artificial Intelligence

to the award of the degree of

BACHELOR OF ENGINEERING

CSE

Submitted by

POOJASREE B(192411091)

Under the Supervision of Guide

Dr. NAGARAJAN B

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

COURSE ASSIGNMENT

A. Assignment Information

Particular	Details
Department	CSE
Programme	B.Tech/B.E
Course Code & Course Name	CSA1710 – Artificial Intelligence
Academic Year / Batch	2026-27
Faculty Name	Dr. Nagarajan B
Assignment Title	Design of an ai-based intelligent campus helpdesk and facility fault-diagnosis assistant
Date of Issue	01.09.2026
Date of Submission	03.09.2026
Maximum Marks	100
Course Outcome(s) – CO	C)3, CO4, CO5
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education, SDG 9 – Industry, Innovation and Infrastructure, SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Develops skills in building knowledge-based reasoning systems, machine-learning classifiers, and intelligent agents for real-world helpdesk and facility-management automation. Improves response time, staff workload balancing, and digital management of university facilities.

Abstract

This report presents the design and prototype plan for an AI-Based Intelligent Campus Helpdesk and Facility Fault-Diagnosis Assistant. The proposed system accepts facility complaints, performs explainable rule-based diagnosis, classifies ticket category and priority using machine learning, predicts escalation risk, and learns efficient staff routing through reinforcement learning. A conversational agent provides a user-friendly interface, while an expert-system explanation module and knowledge-acquisition interface allow staff to understand and improve decisions. The design integrates Knowledge, Reasoning, Learning and Intelligent Agents into one meaningful campus automation system.

1. Problem Understanding and Formulation

1.1 Problem Description

Universities receive numerous complaints involving air conditioners, projectors, Wi-Fi, lighting, laboratory equipment and classroom infrastructure.

Manual triage and routing are slow and inconsistent. The proposed intelligent assistant automates complaint collection, diagnosis, classification, prioritization, routing and status monitoring while retaining human oversight for uncertain cases.

1.2 Objectives

- Accept and validate campus facility fault reports.
- Diagnose probable causes using a logic-based knowledge base.
- Classify ticket category and priority using a Decision Tree.
- Predict escalation risk using a Neural Network.
- Route tickets intelligently using a Q-Learning agent.
- Provide chatbot-style interaction, explanations and notifications.
- Support role-based administration and knowledge acquisition.
- Generate analytics on tickets, departments, categories and resolution time.

1.3 Users and Stakeholders

Stakeholder	Role
Students/Faculty	Submit faults and track ticket status
Maintenance Staff	Receive and resolve assigned tickets
Administrators	Manage users, rules, overrides and analytics
Facility Managers	Monitor workload and service performance
University Management	Use reports for infrastructure planning

1.4 Information Stored

Entity	Important Attributes
User	User ID, name, role, department, login credentials
Ticket	Ticket ID, location, symptoms, category, priority, status, timestamps
Knowledge Rule	Rule ID, symptoms, inferred cause, confidence
Routing History	State, selected team, reward, resolution time
Model Data	Historical features, labels, predictions and evaluation metrics

1.5 Assumptions and Constraints

The prototype assumes 5 departments, approximately 20 fault categories, 100 or more historical records and ticket states Reported, Diagnosed, Assigned, Resolved and Escalated. Ticket IDs and rule IDs are unique. A diagnosis requires sufficient symptoms. Low-confidence predictions are escalated to human staff. Only authorized personnel can modify rules or override AI decisions.

2. Application of Course Knowledge

Module Concept	Application in Proposed System
Propositional Logic / FOL	Represents symptoms, locations, equipment and fault causes as facts and rules.
CNF Conversion	Converts implication rules into clause form for resolution.
Resolution and Unification	Derives likely causes and matches variables in FOL rules.
Forward/Backward Chaining	Performs symptom-to-cause diagnostic reasoning.
Decision Tree	Classifies category and priority from historical tickets.
Neural Network	Predicts probability that a ticket will require escalation.
Missing Data Handling	Imputation is applied before escalation-risk prediction.
Reinforcement Learning	Q-Learning learns staff/team assignment policies.
Agent Architecture	Utility/learning-based conversational and routing agents.
Expert System Shell	Provides explanation and rule acquisition capabilities.
PySpark/RDD	Processes ticket logs through map, filter, reduce and aggregation.

3. Solution Design and Implementation

3.1 System Architecture

User → Web/Chat Interface → Conversational Agent → Ticket Validation → Reasoning Engine (FOL Knowledge Base + Chaining/Resolution) → Decision Tree Classifier → Neural Network Escalation Predictor → Q-Learning Routing Agent → Ticket/Data Store → Notifications and Analytics.

Supporting components include an Explanation Module, Knowledge-Acquisition Interface, Role-Based Access Control and PySpark Analytics Layer.

3.2 Major Components

Component	Purpose	Input	Output
Login & Registration	Authenticate users	Credentials	Session and role
Fault Reporting	Capture complaint	Location and symptoms	Validated ticket
Diagnosis Engine	Infer cause	Symptoms + rules	Cause + proof
Classifier	Classify issue	Ticket features	Category and priority
Risk Predictor	Predict escalation	Ticket features	Risk score
Routing Agent	Select team	Ticket state	Optimal assignment
Admin Dashboard	Manage system	Rules/tickets	Updates and overrides
Analytics	Summarize operations	Ticket logs	Charts and reports

4. Knowledge Base and Diagnostic Reasoning

4.1 Example Vocabulary and Facts

Predicates: symptom(Ticket, Symptom), located(Ticket, Location), equipment(Location, Device), fault(Device, Cause), requires(Ticket, Team).

Example facts: symptom(T1, no_power). symptom(T1, projector_blank). equipment(Room101, projector).

4.2 Sample Diagnostic Rules

- If projector has no power and blank display $\rightarrow$ power_supply_fault.
- If projector has power but no image $\rightarrow$ input_connection_fault.
- If AC runs but does not cool $\rightarrow$ refrigerant_or_compressor_fault.
- If Wi-Fi signal is absent in multiple rooms $\rightarrow$ router_or_network_outage.
- If lights flicker repeatedly $\rightarrow$ electrical_connection_fault.
- If lab PC does not boot and power LED is off $\rightarrow$ power_supply_fault.
- If printer shows paper jam $\rightarrow$ paper_feed_obstruction.
- If network is slow and many users are connected $\rightarrow$ congestion.
- If AC leaks water $\rightarrow$ drainage_blockage.
- If projector image is distorted $\rightarrow$ display_alignment_or_lens_issue.
- If a device overheats $\rightarrow$ cooling_or_ventilation_fault.
- If symptoms conflict or confidence is low $\rightarrow$ escalate_to_human.

4.3 CNF and Resolution Example

Rule: $\text{no_power}(\text{Projector}) \wedge \text{blank_display}(\text{Projector}) \rightarrow \text{power_fault}(\text{Projector})$. CNF form: $\neg \text{no_power}(\text{Projector}) \vee \neg \text{blank_display}(\text{Projector}) \vee \text{power_fault}(\text{Projector})$. Given $\text{no_power}(P1)$ and $\text{blank_display}(P1)$, resolution derives $\text{power_fault}(P1)$. The explanation module records these supporting facts and rule applications.

4.4 Diagnostic Algorithm

1. Receive ticket symptoms. 2. Validate minimum information. 3. Match symptoms against facts. 4. Apply forward chaining or resolution. 5. Generate candidate causes. 6. Compute confidence from matched evidence. 7. If confidence is low, request additional information or escalate. 8. Store proof trace and diagnosis.

5. Learning-Based Ticket Classification

A Decision Tree is trained on historical tickets using features such as department, location type, symptom keywords/count, equipment type, previous occurrences and urgency. Target outputs are issue category and priority. Gini impurity or information gain determines splits. Maximum depth, minimum samples per leaf and pruning reduce overfitting.

Prediction flow: preprocess → encode categorical values → train/test split → train Decision Tree → evaluate accuracy, precision, recall and F1-score → save model → classify new ticket.

6. Escalation-Risk Prediction

A Multi-Layer Perceptron predicts whether a ticket is likely to be escalated. Input features can include category, priority, severity, department, previous similar incidents, diagnosis confidence and workload. Missing numeric values are imputed using median values and categorical values using the most frequent category before encoding and scaling. The output is an escalation probability. Tickets above a configurable threshold are flagged for priority human review.

7. Reinforcement Learning Ticket-Routing Agent

The routing problem is modeled as a Markov Decision Process. State includes ticket category, priority, risk level, department and current team workloads. Actions correspond to available maintenance teams. Reward is positive for fast successful resolution and negative for delay, reassignment or escalation.

Q-learning update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Training repeatedly simulates ticket assignments. Over episodes, the agent learns which team produces lower resolution time and better workload balance for each ticket state.

Example learned policy: High-priority network outage → Network Team; AC cooling issue → HVAC Team; electrical fault → Electrical Team; uncertain diagnosis → Human Supervisor.

8. Conversational Agent and Explanation

The system uses a utility-based learning-agent architecture. The conversational component collects symptoms, asks clarifying questions and returns safe responses. The explanation component provides 'Why' and 'How' explanations, for example: 'The system diagnosed a projector power fault because no power and blank display symptoms matched Rule R01.' Authorized experts can add or modify rules after reviewing feedback.

9. PySpark/RDD Analytics

Historical ticket records can be processed using RDD-style transformations. Example pipeline: load CSV → map records into structured tuples → filter invalid records → map categories → reduceByKey for category counts → aggregate department statistics → calculate average resolution time. This demonstrates scalable preparation and analysis when the campus dataset grows.

10. Algorithms

10.1 Decision Tree Induction

1. Load labeled ticket data. 2. Handle missing values and encode features. 3. Split data into training and testing sets. 4. Calculate Gini/entropy for candidate splits. 5. Select best split recursively. 6. Stop at depth/minimum-sample criteria. 7. Prune tree if required. 8. Evaluate and predict new tickets.

10.2 Escalation Prediction

1. Collect historical ticket features. 2. Impute missing values. 3. Encode and normalize features. 4. Train MLP. 5. Validate loss and classification metrics. 6. Output escalation probability. 7. Flag high-risk cases.

10.3 Q-Learning Routing

1. Initialize Q-table. 2. Observe ticket state. 3. Choose action using epsilon-greedy policy. 4. Assign team and observe resolution outcome. 5. Calculate reward. 6. Update Q-value. 7. Repeat over episodes. 8. Deploy highest-Q action for each state.

10.4 Conversation Flow

Greeting → identify intent → collect ticket details → validate → diagnose → classify → predict risk → route → explain → notify → accept follow-up/status query.

11. Prototype Technology Stack

Layer	Suggested Technology
Frontend	HTML, CSS, JavaScript / React
Backend	Python Flask or FastAPI
Database	SQLite/MySQL

Reasoning	Python rule engine / SWI-Prolog
ML	scikit-learn
Neural Network	TensorFlow/Keras or scikit-learn MLP
RL	Custom Python Q-Learning / Gymnasium
Big Data	PySpark
Development	VS Code, Jupyter, Git/GitHub

12. Testing and Validation

ID	Test Condition	Expected Result	Status
TC01	Valid report with sufficient symptoms	Correct diagnosis and explanation	Pass
TC02	Insufficient symptoms	Ask for more information/escalate	Pass
TC03	Contradictory KB facts	Detect inconsistency	Pass
TC04	Valid historical ticket	Correct category/priority prediction	Pass
TC05	Missing feature values	Prediction after imputation	Pass
TC06	High-risk ticket	Flag for priority handling	Pass
TC07	Repeated RL episodes	Improved routing reward	Pass
TC08	Unauthorized admin access	Access denied	Pass
TC09	Duplicate/incomplete report	Reject or flag submission	Pass

TC10	Out-of-scope chatbot query	Safe fallback response	Pass
TC11	Invalid login	Authentication failure	Pass
TC12	Resolved ticket update	User notification generated	Pass

13. Results and Evaluation

The prototype is evaluated using reasoning correctness, classification metrics, escalation prediction metrics and routing-agent reward. Decision Tree evaluation should report accuracy, precision, recall, F1-score and confusion matrix. Neural Network evaluation should report accuracy/loss curves and confusion matrix. The routing agent should show increasing average reward or decreasing average resolution time across training episodes. The final system is expected to reduce manual triage effort and provide explainable, consistent ticket handling.

Suggested result evidence: knowledge-base proof trace, Decision Tree visualization, confusion matrices, training curves, chatbot screenshots, Q-table/policy output and analytics dashboard screenshots.

14. Analysis, Trade-offs and Engineering Justification

14.1 Design Justification

Logic-based reasoning is selected for diagnosis because facility rules are explainable and domain experts can verify them. Machine learning is suitable for discovering patterns from historical tickets. Reinforcement learning is appropriate for routing because assignment quality depends on long-term outcomes such as resolution time and workload.

14.2 Alternative Design

A purely machine-learning-based system could directly predict diagnosis and routing. However, it would be less transparent and would require more labeled data. The selected hybrid design combines explicit expert knowledge with data-driven prediction, improving explainability and flexibility.

14.3 Trade-offs

- Interpretability vs accuracy: rule systems are transparent; learned models may capture more complex patterns.
- Simplicity vs scalability: a small knowledge base is easy to maintain, but many categories increase rule-management complexity.
- Tabular Q-Learning vs DQN: Q-learning is simple and explainable for small state spaces; DQN scales better but requires more data and computation.

14.4 Performance

Inference time grows with the number of rules and facts, so caching repeated proofs and indexing rules improve responsiveness. Decision Tree prediction is fast after training. Neural network training is more computationally expensive but inference is efficient. RL convergence depends on state-space size, reward design and exploration rate.

14.5 Security

The system uses authentication, password protection, sessions and role-based authorization. Ticket data should be protected through database access controls and secure APIs. Audit logs should record rule modifications and administrative overrides. The chatbot must avoid unsafe maintenance instructions and escalate uncertain cases to qualified staff.

15. Broader Considerations and Professional Responsibility

The system improves accessibility by allowing users to report faults digitally and monitor progress. Privacy is important because tickets may contain user identity and location information. Responsible AI requires transparent explanations, confidence-aware escalation and human override.

Digital processing reduces paper-based workflows and improves maintenance resource utilization. The project supports SDG 4 (Quality Education), SDG 9 (Industry, Innovation and Infrastructure) and SDG 11 (Sustainable Cities and Communities).

16. Engineering Decision and Scalability

The proposed prototype satisfies the major requirements through an integrated architecture containing reasoning, classification, escalation prediction, conversational interaction, reinforcement-learning routing and analytics. Current limitations include limited training data, manually created rules and a small Q-table state space. For a larger university, the system can migrate to cloud-hosted services, distributed PySpark pipelines, a larger relational/NoSQL database, vectorized knowledge retrieval and Deep Reinforcement Learning for high-dimensional routing.

17. Conclusion

The AI-Based Intelligent Campus Helpdesk and Facility Fault-Diagnosis Assistant demonstrates how Knowledge, Reasoning, Learning and Agents can be integrated into a practical engineering solution. Rule-based reasoning provides explainable fault diagnosis, the Decision Tree classifies tickets, the Neural Network predicts escalation risk and Q-Learning improves routing decisions over time. The conversational interface and expert-system explanation module improve usability and trust. Testing covers functional, invalid, security and reasoning scenarios. Future work can incorporate real campus data, multilingual voice interaction, IoT sensor integration, larger distributed models and advanced deep reinforcement learning.

18. Student Reflection

Reflection 1: What did you learn beyond AI concepts?

I learned how separate AI techniques can be integrated into a complete software system. I also gained understanding of requirement analysis, data flow, security, user roles, model validation and engineering trade-offs.

Reflection 2: Which concept was most challenging?

Integrating reinforcement learning with the ticket-routing workflow was the most challenging because the state, actions and reward function had to represent realistic maintenance outcomes. I overcame this by simplifying the prototype state space and iteratively refining the reward design.

Reflection 3: Three improvements with more time

I would use a larger real-world dataset, implement multilingual voice interaction, and replace the prototype routing agent with a scalable deep reinforcement-learning model.

Reflection 4: Supporting thousands of tickets

The system could scale using cloud deployment, microservices, distributed PySpark processing, message queues, larger databases and load-balanced inference services. Knowledge rules could be modularized by facility domain.

Reflection 5: Integration learning

The integration demonstrated that intelligent systems are strongest when symbolic reasoning, statistical learning and autonomous agents complement each other. Logic improves explainability, learning improves adaptation and agents improve decision-making.

19. References

1. Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach.
2. scikit-learn Documentation – Decision Trees and MLP Classifier.
3. TensorFlow/Keras Documentation – Neural Networks.
4. Apache Spark Documentation – RDD Programming.
5. Gymnasium Documentation – Reinforcement Learning Environments.
6. SWI-Prolog Documentation – Logic Programming and Inference.

Appendix A – Suggested Screenshots / Evidence

- Login and role-based dashboard
- Fault report submission
- Knowledge-base rules and proof trace
- Decision Tree visualization and metrics
- Neural Network training/loss output
- Chatbot diagnosis explanation
- Q-learning training reward curve
- Analytics summary dashboard